

Procedure acquisition and reuse: a bounded local comparison

Ancillary study: alignment-farm

11 September 2026

Abstract

When does repeated use justify storing a procedure in parameters rather than consulting explicit experience? We compare twelve checked demonstrations, a single generated lesson, and a fixed supervised LoRA update on an unfamiliar tool-argument protocol. We distinguish exploratory execution diagnostics from a frozen evaluation on new random identifiers and an unobserved condition combination. All branches use the same local 4B model and native runtime. The adapter recalled all twelve acquisition calls but passed 26/64 new-input tasks, versus 41/64 with examples and 64/64 with the supplied procedure. A defective generated lesson passed 0/64. No quality-matched repayment was established. These are narrow feasibility observations, not estimates of the best achievable learning performance.

1 Question and prior work

The question concerns useful generated behavior and acquisition cost, not whether training loss decreases. SEAL learns adaptation-data generation using post-update performance; our fixed writer-free update does not reproduce that outer loop [6]. Procedural Memory Distillation uses cross-episode memory to condition a self-teacher and distills it into a changing policy; internalization of procedural memory is already prior art [3]. PERK meta-learns a LoRA initialization for encoding context and later reasoning, unlike our ordinary randomly initialized adapter [4].

ACE motivates explicit contextual playbooks but uses a richer incremental process than our single lesson-generation call [5]. Beyond Perplexity motivates testing generated behavior after removing evidence, while separating that behavior from likelihood gains [2]. Amortized inference work already moves computation from future uses into training, including tool-use settings [1]. We do not replicate these algorithms or claim that a weak fixed adapter tests their limits. The contribution is an auditable local comparison under a small, explicitly bounded recipe.

2 Development and frozen protocol

An item arrives with a mode (amber/blue) and priority (normal/urgent). Amber selects tool `dax` and uppercases the item; blue selects `wug` and preserves it. Normal appends `-2`; urgent appends `-9`. Success requires the exact tool call with one space between tool and argument. A deterministic oracle checks the call contract; no real external service is invoked.

Development originally included string reversal. The inherited Docker attempt stopped after 27 responses and an HTTP 500. A later MLX draft needed API repairs. These failures and original source snapshots remain preserved. The Docker and native artifacts are not established to be the

same checkpoint, so no cross-runtime causal claim is made. Native component diagnostics subsequently scored 24/24 each for routing, conditional transformation, suffix selection, and formatting supplied intermediate values; full instructions scored 21/24. Perfect execution was therefore replaced as a gate by a measured supplied-rule reference. Development strings are excluded from the frozen evaluation.

2.1 Acquisition information and branches

Twelve researcher-generated, oracle-checked demonstrations cross four words (cat, fern, oak, reed) with amber/normal, blue/normal, and blue/urgent. Amber/urgent is withheld. The intended rule’s separability is an inductive assumption: finite demonstrations do not uniquely identify it. The branches retain:

- No acquisition: only the common task/output schema and current input.
- Examples: all twelve checked interactions in every prompt; no retrieval selection.
- Lesson: one greedy generation from those interactions, requested to be at most 100 words, with a 256-token ceiling. No manual repair or selection.
- Oracle rule: the original researcher-supplied procedure, a privileged diagnostic.
- Adapter: only the updated LoRA parameters, with no examples, lesson, or procedure in the evaluation prompt.

The generated lesson omits the tool names and case transformation, and contradicts itself about suffixes. It is retained unchanged. Its failure should not be generalized to well-constructed explicit lessons or ACE. The adapter receives the same twelve calls as supervised targets, without oracle-derived augmentation.

2.2 Model, training, and evaluation

All branches use the local Qwen3-4B-Instruct-2507 bf16 checkpoint, verified against prior native file hashes. MLX-LM is pinned to commit `86b48c461feebf87c58788655b7e57b5574b9e6d`. Query and value projections in the last eight blocks receive rank-eight LoRA, scale two, dropout zero. AdamW uses learning rate 0.0005 and zero weight decay. One seed (17), eight shuffled epochs, batch size one, and 96 steps are fixed before evaluation. Loss covers assistant answer and EOS only, with an asserted exact prompt-token boundary. No hyperparameter search or checkpoint selection occurs.

After the final checkpoint is fixed, Python Random(20260911) generates sixteen previously unused lowercase identifiers, four each of lengths 4, 6, 8 and 10. Crossing all four conditions yields 64 paired tasks, including 16 withheld-combination tasks. Random identifiers extend beyond the natural acquisition words, so this tests a demanding character-transfer setting. It does not isolate all possible forms of generalization.

Five branches rotate within each input across two passes: 640 calls, 128 measured uses per branch. Each call uses fresh generation caches, greedy decoding and a 96-token ceiling; no retries or output repair. Initial/final adapters switch in memory. Checks verify unchanged base parameters and exact restored logits. Training recall is measured separately. The two passes are repetitions, not independent samples; the experimental diversity is sixteen identifier clusters, not 128 independent tasks per branch.

3 Results and costs

Representation	All /64	Seen /48	Withheld /16
No acquisition	0	0	0
Examples	41	39	2
Generated lesson	0	0	0
Supplied rule	64	48	16
Adapter	26	22	4

Table 1: First-pass exact-call success on 64 new-input tasks.

The adapter recalls 12/12 training calls but succeeds on only 26/64 new-input calls. Examples achieve 41/64; the supplied rule achieves 64/64. The generated lesson and no-acquisition reference each achieve 0/64. This distinguishes fitting the acquisition examples from reliable transfer.

Second-pass exact outputs match the first pass on No acquisition 64/64, Examples 64/64, Generated lesson 64/64, Supplied rule 64/64, Adapter 64/64. These repetitions add cost measurements, not independent accuracy evidence.

Representation	Input tokens	Output tokens	Use seconds
No acquisition	8,392	814	40.37
Examples	37,192	1,176	69.03
Generated lesson	16,712	968	50.16
Supplied rule	15,176	1,204	53.41
Adapter	8,392	1,128	47.76

Table 2: Measured generation costs for 128 uses per branch; excludes acquisition and switching.

The fixed update took 13.16 seconds including its final hash and adapter save, processing 6,568 input positions and supervising 616 answer/EOS tokens. The adapter occupies 2,624,893 bytes. Lesson generation took 2.03 seconds (274 input and 66 output tokens). Peak MLX allocation over the run was 9.64 GB; total wall time including provenance and checks was 302.86 seconds.

The adapter has lower useful performance than examples, so a faster or shorter adapter call would not establish a storage advantage at comparable quality. No repayment relative to examples is established in this workload. A comparison against the failed generated lesson cannot rescue a broad parametric-storage claim. The machine-readable analysis includes cumulative measured use costs and explicitly labeled linear arithmetic projections; projections with lower adapter accuracy are ineligible as quality-matched break-even claims.

Base-parameter hashes remained unchanged and restored probe logits matched exactly after adapter resets. The final checkpoint was fixed before evaluation; no fitting followed the observed failures.

4 Interpretation and limits

A fixed small supervised adapter can change generated behavior without acquiring a reliably transferable procedure. Training recall and new-input execution must be separated. A failed single generated lesson is evidence about lesson construction in this recipe, not proof of a parametric

advantage. The supplied rule exposes the extra knowledge available to that diagnostic, and the examples branch retains the actual acquisition evidence at use time.

This is one model, one seed, one synthetic task, and one fixed update budget. There is no locality or subsequent-learning test, and no claim of retention across unrelated updates. Exact-call success measures a narrow tool contract rather than general agent competence. The composition holdout depends on a separability assumption. The model’s pretrained knowledge of casing is unavoidable and disclosed. A different training budget, writer, architecture, or identifier distribution could change the outcome; this study deliberately does not search until a parameter-storage win appears.

The cost ledger separates model generation, training, preparation, switching and verification. No persistent prefix cache is retained. Repetitions use the same 64 tasks; larger cumulative curves are linear projections, not further measurements. Human research/design labor, energy, dollars, production model-startup and operational infrastructure are unmetered. Small differences in wall time on a shared laptop are descriptive, not stable deployment speed estimates. Cost comparisons are meaningful only alongside useful performance.

5 Reproduction and evidence

The repository README links the protocol, analysis and raw evidence. The protocol was committed before evaluation (0aae909). Install the pinned adaptation environment and the identified local checkpoint using the README instructions. Run the acquisition script with a fresh output directory. Existing directories are rejected. The cooperative resource limits are twenty minutes and 24 GB peak MLX allocation; a blocked native call is not forcibly interrupted by these checks.

The run directory preserves protocol, code, acquisition/evaluation cases, exact training tokens, lesson, all raw calls, events, final adapter and checksums. The analysis verifies checksums and request coverage before recomputing results. Use `scripts/analyze_acquisition.py` with the run directory. Unit tests cover independent task fixtures, original-pilot preservation, split integrity and component scoring. Model weights are excluded from Git; the small final adapter is explicitly included. The paper’s build instructions record the compiler and output validation. External submission is outside this study.

References

- [1] Amortizing Intractable Inference in Large Language Models, 2023. arXiv:2310.04363v2.
- [2] Beyond Perplexity: A Behavioral Evaluation Framework for Deployment-Memory Claims in LLM Test-Time Training, 2026. arXiv:2607.00368v1.
- [3] Ye Liu et al. Procedural Memory Distillation: Online Reflection for Self-Improving Language Models, 2026. arXiv:2607.01480v1.
- [4] PERK: Long-Context Reasoning as Test-Time Learning, 2025. arXiv:2507.06415v3.
- [5] Qizheng Zhang et al. Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models, 2025. arXiv:2510.04618v3.
- [6] Adam Zweiger et al. Self-Adapting Language Models, 2025. arXiv:2506.10943v2.